

空气质量四分类任务：经典与量子机器学习模型设计与性能分析

1. 概述

结合机器学习原理，使用代码实现对数据的预处理，获取训练集和测试集数据的全部特征。结合数据探索的结果，使用 VQNet 构建经典神经网络模型，完成上述数据的空气质量四分类任务。其中经典机器学习模型参数量不超过 200。运行该代码可以输出准确率，平均 F1 分数 2 个值，并保存到 txt 后缀的文本文件中。使用 VQNet 以及 QPanda，结合量子计算设计量子机器学习模型，完成上述数据的空气质量四分类任务，模型参数量(量子参数量+经典参数量)不超过 200。运行该代码可以输出准确率，平均 F1 分数 2 个值，并保存到 txt 后缀的文本文件中。

2. 数据探索与预处理

(1)读取数据文件

使用 `pandas.read_csv()` 读取指定的 .csv 文件。

输出数据的维度（行数和列数）、所有列的名称、每列的缺失值数量。

(2)缺失值检测

检查是否存在包含缺失值的行。

如果有缺失值，打印出具体的行内容；否则提示没有缺失值。

(3)处理数值特征中的异常值

选取所有类型为 `float64` 和 `int64` 的数值型列。

统计这些列中负值的总数（替换前）。

使用 `.clip(lower=0)` 将所有负值替换为 0。

再次统计负值的总数（替换后）以确认替换结果。

(4)输出数值统计信息

针对所有数值型特征，输出如下统计信息：

均值、标准差、最小值、25%分位数、中位数（50%）、75%分位数、最大值
方差（通过 `.var()` 手动计算，并添加到 `describe()` 结果中）

(5)保存处理后的数据

以原始文件名为基础，生成一个新文件名（如 `train_data_processed.csv`）。

将预处理后的数据保存为新 CSV 文件，供后续模型训练或分析使用。

3. 经典神经网络模型

3.1 算法原理

该算法基于多层前馈神经网络（Feedforward Neural Network），旨在对输入的空气品质相关环境特征进行建模，并预测空气质量等级（共 4 类）。训练过程中使用交叉熵损失函数衡量预测与真实标签之间的差异，并利用 Adam 优化器进行梯度下降更新模型参数。

该任务属于监督学习中的多分类问题，输入为 9 维环境特征，输出为 4 类空气质量标签。

3.2 实现细节

(1)数据预处理与张量转换

首先通过 `load_data()` 函数完成数据的加载、标准化和编码：

```
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
le = LabelEncoder()
y_train = le.fit_transform(y_train)
```

使用 `StandardScaler` 对特征标准化，使特征均值为 0、方差为 1；

使用 `LabelEncoder` 将类别标签（字符串）转为整数（0~3）；

所有数据随后被封装成 `QTensor` 类型张量，便于后续在 `VQNet` 中计算与自动求导。

(2)神经网络模型构建

模型 `AirQualityNN` 继承自 `pyvqnet.nn.Module`，结构如下：

```
self.fc1 = nn.Linear(9, 7)      # 输入层 (9 维特征) -> 隐藏层 (7 个神经元)
self.relu = nn.ReLU()          # ReLU 激活函数
self.fc2 = nn.Linear(7, 4)      # 隐藏层 -> 输出层 (4 分类)
self.softmax = nn.Softmax(axis=1) # softmax 将输出转换为概率
```

输入层接受 9 维环境特征；

隐藏层包含 7 个神经元，使用 `ReLU` 激活函数引入非线性；

输出层为 4 个神经元，对应四类空气质量标签；

`softmax` 函数将输出归一化为分类概率。

前向传播函数如下：

```
def forward(self, x):
    x = self.fc1(x)
    x = self.relu(x)
    x = self.fc2(x)
    x = self.softmax(x)
    return x
```

(3)模型训练过程

在 `model_train()` 函数中，构建损失函数、优化器并执行训练循环：

```
model = AirQualityNN()
criterion = CrossEntropyLoss()
optimizer = Adam(model.parameters(), lr=0.005)
```

训练循环如下：

```
for epoch in range(num_epochs):
    for inputs, labels in data_generator(..., batch_size=20, shuffle=True):
        optimizer.zero_grad()
        outputs = model(inputs)
        loss = criterion(labels, outputs)
        loss.backward()
        optimizer.step()
```

每轮训练分批读取数据，进行前向传播和反向传播；

使用 `CrossEntropyLoss` 评估预测概率与实际类别的差异；

使用 `Adam` 优化器更新模型参数；

每轮结束后评估模型在测试集上的表现，并记录训练与测试损失。

训练损失与测试损失平均值记录如下：

```
train_loss = running_loss / 200
test_loss = test_loss / 200
```

(4)模型评估与指标计算

训练完成后，调用 `model_test()` 加载模型并进行评估：

```
model.load_state_dict(pyvqnet.utils.storage.load_parameters('air_quality_model.pth'))
model.eval()
```

在无梯度模式下进行预测：

```
outputs = model(inputs)
predicted = outputs.argmax(dim=1)
```

预测结果通过 `accuracy_score` 和 `f1_score` 衡量：

```
accuracy = accuracy_score(y_true, y_pred)
f1 = f1_score(y_true, y_pred, average='macro')
```

最终将评估结果保存至文件：

```
with open('model_evaluation.txt', 'w') as f:
    f.write(f'Accuracy: {accuracy}\n')
    f.write(f'Average F1 Score: {f1}\n')
```

3.3 性能评估

(1) 准确率与 F1 分数

准确率 (Accuracy) : 0.937

F1 分数 (F1 Score) : 0.9099

说明模型具有优秀的整体分类能力，对四个类别均有较好的区分效果。

(2) 损失变化与收敛情况

从训练日志可见：

第 1 轮训练损失较高 (Train Loss $\approx$ 1.07)，测试损失为 0.2283。

第 10 轮开始后，训练损失稳定在 0.80 左右，测试损失稳定在 0.20 左右。

表明模型在前几轮迅速收敛，随后保持较好的稳定性，没有明显过拟合或欠拟合迹象。

4. 量子机器学习模型

4.1 算法原理

构建了一个基于经典-量子混合神经网络 (Hybrid Classical-Quantum Neural Network) 的模型，用于实现空气质量的四分类任务。该方法利用经典神经网络的特征提取能力与量子线路的非线性映射能力，在保持模型参数可控的同时提升分类性能。

模型核心基于变量子算法 (Variational Quantum Algorithm, VQA)，通过训练参数化量子电路，使输出接近最优预测结果。

4.2 实现细节

(1) 数据加载与预处理

```
def load_data():
    # 标准化特征，编码标签，转换为张量
    scaler = StandardScaler()
    X_train = scaler.fit_transform(X_train)
    X_test = scaler.transform(X_test)

    le = LabelEncoder()
    y_train = le.fit_transform(y_train)
    y_test = le.transform(y_test)

    # 转换为 QTensor
    X_train_tensor = QTensor(X_train, dtype=kfloat32)
    y_train_tensor = QTensor(y_train, dtype=kint64)
```

标准化：消除特征量纲差异。

标签编码：将类别标签（如"Good", "Moderate"）转换为整数。

张量化：使用 QTensor 适配量子框架计算。

(2)量子线路设计

```
class VariationQuantumCircuit():
    def pqctest(inputs, params):
        num_of_qubits = 4
        layers = 4
        # 编码层: RY(inputs[i])
        for i in range(num_of_qubits):
            circuit << pq.RY(qubits[i], inputs[i])
        # 变分层: RZ-RY-RZ + CNOT 环
        for layer in range(layers):
            for i in range(num_of_qubits):
                circuit << pq.RZ(qubits[i], params[param_idx])
                # ... 省略 RY、RZ
            circuit << pq.CNOT(qubits[0], qubits[1]) # 环状纠缠
```

编码方式：输入特征通过 RY 门编码到 4 个量子比特的量子态。

变分结构：每层包含参数化的 RZ-RY-RZ 门和环状 CNOT 纠缠，增强模型表达能力。

参数数量：4 层 × 4 量子比特 × 3 门/层 = 48 个可训练参数。

(3)混合模型结构

```
class AirQualityQuantumNN(Module):
    def __init__(self):
        self.classical_preprocess = nn.Sequential(
            Linear(9, 6), ReLU(), Linear(6, 4)
        )
        self.quantum_layer = QuantumLayer(VariationQuantumCircuit.pqctest, 48)
        self.post_classifier = Linear(4, 4)
```

经典预处理：9 → 6 → 4 的全连接层，ReLU 激活。

量子层：调用自定义量子线路，输入 4 维，输出 4 个测量概率（通过 probs_measure 获取）。

分类层：Linear(4, 4)将量子输出映射到 4 个类别。

(4)训练过程

```
criterion = CrossEntropyLoss()
optimizer = Adam(model.parameters(), lr=0.005)
for epoch in range(num_epochs):
    for inputs, labels in data_generator(...):
        outputs = model(inputs)
        loss = criterion(labels, outputs)
        loss.backward()
        optimizer.step()
```

损失函数：交叉熵损失（适用于多分类）。

优化器：Adam，学习率 0.005。

批训练：batch_size=20，数据分批次加载。

(5)评估指标

```
accuracy = accuracy_score(y_true, y_pred)
```

```
f1 = f1_score(y_true, y_pred, average='macro')
```

准确率：正确预测的比例。

宏平均 F1：考虑类别不平衡的综合指标。

4.3 性能评估

(1)损失曲线与收敛性

训练损失：

从初始的 1.1336 快速下降至 0.1240（第 50 epoch），整体下降幅度显著（约 89%）。

前 5 个 epoch 内损失急剧下降（从 1.13 → 0.20），表明模型快速捕捉到数据的主要模式。

后续 epoch 损失平缓下降并趋于稳定（20~50 epoch 波动在 0.12~0.14 之间），显示模型逐渐收敛。

测试损失：

从 0.1896 下降至 0.0345（降幅约 82%），与训练损失同步下降，未出现明显过拟合。

测试损失始终低于训练损失（如第 50 epoch：0.0345 vs. 0.1240）

收敛性：

训练与测试损失在约 30 epoch 后基本稳定（波动幅度<0.01），表明模型在 50 epoch 内充分收敛。

最终测试损失接近 0.03，验证了模型的高效性。

(2)分类性能

准确率：95.2%，显著高于随机猜测（25%，假设 4 类均衡）。

宏平均 F1 分数：0.93，说明模型在各类别上均表现优异，无明显类别偏好。

性能亮点：

高 F1 分数（接近准确率）表明模型在召回率和精确率间取得良好平衡。

结果可能反映量子特征提取（如 RY 门编码、CNOT 纠缠）有效捕获了空气质量数据的非线性关系。

5. 性能对比与分析

5.1 性能对比

量子模型在分类任务中展现出显著优势。其准确率达到 95.2%，比经典模型的 93.7% 高出 1.5 个百分点；宏平均 F1 分数为 0.930，也优于经典模型的 0.910，表明量子模型在召回率与精确率的平衡上更优。此外，量子模型的损失值远低于经典模型：最终训练损失为 0.124（经典模型为 0.797），测试损失仅为 0.0345（经典模型为 0.2012），两者差距分别达到 84.4% 和 82.8%。量子模型在训练初期（前 5 个 epoch）即快速收敛，损失下降超过 80%，而经典模型全程呈现缓慢下降趋势，最终仍未完全收敛。

5.2 差异分析

量子模型通过混合架构结合了经典预处理与量子特征提取：经典预处理层（9→6→4 的全连接网络）对原始数据进行降维，适配量子层输入。量子层采用 4 量子比特的变分线路，包含 4 层参数化旋转门（RZ-RY-RZ）和环状 CNOT 纠缠结构，共 48 个可训练参数，通过量子态的叠加与纠缠提取非线性特征。经典分类层（4→4）完成最终分类。

相比之下，经典模型仅使用简单的全连接网络（9→7→4），依赖 ReLU 激活函数引入非线性，缺乏高阶特征提取能力。量子模型的参数复杂度更高，但得益于量子纠缠的隐式正则化效果，未出现明显过拟合。

训练动态差异：

初始阶段：量子模型的测试损失初始值（0.19）低于经典模型（0.23），表明量子层的初始化更贴近数据分布。

收敛速度：量子模型在前 5 个 epoch 内损失骤降 80%，而经典模型全程仅下降约 25%，反映量子梯度优化方向更高效。

泛化表现：量子模型的测试损失始终低于训练损失（如最终测试损失 0.0345 vs. 训练损失 0.124），而经典模型的测试损失与训练损失接近（0.2012 vs. 0.797），暗示量子模型泛化能力更强，经典模型可能存在轻微过拟合。